Practical tutorial for `linstabpy`

This tutorial is for new users who want to run the code before changing it. It walks through the installation check, the Martini example family, the Song 2023 validation case, and the pattern for adapting a shipped case to a new configuration.

1. What you will do

At the end of this tutorial you will have:

- installed or checked the local `linstabpy` runtime,
- run a fast smoke example before starting expensive examples,
- run each Martini example from its paired Python and JSONC files,
- understand where reusable code, examples, studies, and validation assets live,
- use the single example runner for serial, parity, and MPI-safe runs,
- run the MATLAB-parity comparison for the Martini examples,
- run the Song 2023 baseline and its JSONC-driven validation matrices,
- learned how to copy a case and adjust parameters for a new study.

Note

Every blue box contains terminal commands to type from the repository root unless another directory is stated.

Warning

When you see `/path/to/linstabpy`, replace it with the real path to your local checkout. In this repository, use `.venv/bin/python`. Do not replace these commands with `uv run` for PETSc/SLEPc runs.

2. Repository and runtime check

Move to the repository root and inspect the environment before installing anything.

Command

```
cd /path/to/linstabpy
bash diagnose.sh
```

If the diagnostic reports missing system packages on Linux, install or load them outside this tutorial. The installer is written to avoid silent system changes by default.

Install the project runtime:

Command

```
bash install.sh
bash install.sh --check
```

The install script creates `.venv/` and installs PETSc/SLEPc support for the project. After installation, verify the Python used by the examples:

Command

```
.venv/bin/python --version
.venv/bin/python -c "import linstab2d; print(linstab2d.__file__)"
```

Note

The import package is `linstab2d`. The repository and distribution name are `linstabpy`. Keep this distinction in mind when reading commands and imports.

3. The code pattern

Most runnable studies in this repository follow a paired-file pattern:

Expected output

```
examples/example_1_blasius.py
examples/example_1_blasius.jsonc
```

The Python file contains the workflow. The JSONC file contains the run parameters. For first modifications, change the JSONC file or copy the pair. Avoid hardcoding study parameters inside the package code.

The examples also follow a cache pattern:

Command

```
.venv/bin/python -m examples.example_1_blasius
```

If the matching `.npz` already exists, the script usually loads it and plots. To recompute, remove the matching `.npz` and rerun:

Command

```
rm -f examples/example_1_blasius.npz
.venv/bin/python -m examples.example_1_blasius
```

Warning

The full examples can be memory- and time-intensive. Start with the smoke run and the cached outputs before deleting large `.npz` artifacts.

Where files belong

The top-level directories have distinct roles. Keep reusable package code, runnable examples, study campaigns, and validation data separated:

Expected output

```
src/linstab2d/      importable Python package
examples/           runnable Martini examples; generated .npz/.png ignored
studies/case1/      SWBLI study: configs, data/baseflow, reference, scripts
studies/case2/      TCFD campaign: configs, data/baseflow, scripts, docs
validation/         reproducibility harnesses, fixtures, reports, references
validation/martini2024/tools/
                    MATLAB-to-Python conversion and parity tooling
scripts/            repository guardrails and maintenance utilities
install_tester/     thin install/diagnose/benchmark harness
tutorial/           tutorial source and generated PDF
```

Do not put reusable solver code under `studies/` or `examples/`. Put reusable code under `src/linstab2d/`, runnable examples under `examples/`, case-specific workflows under `studies/`, and reference/validation assets under `validation/`. Generated outputs such as example `.npz`/`.png` files, study `output/` directories, caches, and build metadata are ignored and should not define the repository structure.

Use one example runner

`run_all_examples.sh` is the single documented runner. It covers serial example passes, cache cleanup, Martini parity, and MPI-safe launches. Direct module execution is still useful for local debugging, but use the runner for acceptance checks and MPI runs.

Command

```
bash run_all_examples.sh --list
bash run_all_examples.sh
bash run_all_examples.sh --fresh --example 1 --skip-parity
bash run_all_examples.sh --clean --example 1 --skip-parity
```

For MPI runs, enable `-mpi`. If `-n/-ranks` is omitted, the runner starts from half of the detected physical cores. The runner also sets one OpenMP/BLAS thread per rank unless those variables were already set by the caller.

Command

```
bash run_all_examples.sh --mpi -m examples.example_1_blasius_smoke
bash run_all_examples.sh --mpi -n 10 -m examples.example_3_streaky_bl
bash run_all_examples.sh --mpi -n 1 -m examples.example_1_blasius_smoke -- -eps_monitor
```

Note

The previous separate MPI wrapper has been folded into `run_all_examples.sh`. Override the MPI launcher or default rank count with `LINSTABPY_MPIEXEC` and `LINSTABPY_MPI_RANKS` when a machine needs explicit runtime paths.

4. First run: Blasius smoke test

The fastest first test uses the same physics path as the Blasius example but on a smaller grid.

Command

```
.venv/bin/python -m examples.example_1_blasius_smoke
```

Expected outputs:

Expected output

```
examples/example_1_blasius_smoke.npz
examples/example_1_blasius_smoke.png
```

The smoke JSONC file is:

Expected output

```
examples/example_1_blasius_smoke.jsonc
```

Use it to understand the basic blocks: flow parameters, grid size, sponge, forcing mask, boundary-condition recipe, assembly backend, and solver settings.

5. Martini example family

The Martini examples are the first complete tour through the library. They exercise derivative validation, resolvent analysis, temporal eigenvalues, spatial eigenvalues, direct modes, adjoint modes, 2-D operators, and axisymmetric operators.

Expected output

```
0  example_0_curvilinear_validation  derivative and curvilinear-grid check
1  example_1_blasius                 Blasius boundary-layer resolvent
2  example_2_jet                     axisymmetric jet resolvent
3  example_3_streaky_bl              temporal and spatial stability
4  example_4_cavity                  direct and adjoint cavity EVP
5  example_5_paraboloid              curvilinear axisymmetric resolvent
```

5.1 Example 0: curvilinear derivative validation

This example contains no flow physics. It checks SBP finite-difference derivatives on a conformal mapping.

Command

```
.venv/bin/python -m examples.example_0_curvilinear_validation
```

Main knobs:

File excerpt

```
"nx": 50,
"ny": 50,
"fd_order": 4,
"alpha_filter": 0.0,
"conformal_scale": 0.2
```

Expected outputs:

Expected output

```
examples/example_0_curvilinear_validation.npz
examples/example_0_curvilinear_validation.png
examples/example_0_curvilinear_validation_baseflow.png
examples/example_0_curvilinear_validation_support_error.png
```

5.2 Example 1: Blasius resolvent

This is the standard boundary-layer resolvent example. Use the smoke version first, then the full example.

Command

```
.venv/bin/python -m examples.example_1_blasius_smoke
.venv/bin/python -m examples.example_1_blasius
```

Main knobs:

File excerpt

```
"Re": 6e5,
"Ma": 0.3,
"nx": 300,
"ny": 100,
"omega": 60.0,
"n_modes": 3,
"forcing_zero_vars": ["T", "rho", "w"]
```

The boundary condition is Dirichlet on all sides for `uvwT`:

File excerpt

```
"bc": {
  "penalty": 1e4,
  "recipe": [
    {"type": "dirichlet", "borders": "lrbt", "variables": "uvwT"}
  ]
}
```

5.3 Example 2: axisymmetric jet resolvent

This case uses the axisymmetric model branch.

Command

```
.venv/bin/python -m examples.example_2_jet
```

Main knobs:

File excerpt

```
"St": 0.39,
"m": 0,
"model": "axy",
```

```
"coarsen": 1,
"fd_order": 4
```

Use `coarsen` for faster exploratory runs. Keep `model: "axy"` for axisymmetric cases.

5.4 Example 3: streaky boundary layer

This example runs both temporal and spatial stability branches.

Command

```
.venv/bin/python -m examples.example_3_streaky_bl
```

Main knobs:

File excerpt

```
"x_periodicity": true,
"temporal_targets": [[0.59, 0.02], [0.626, -0.005]],
"temporal_quotas": [2, 8],
"n_modes": 50,
"ncv": 150,
"run_temporal": true,
"run_spatial": true
```

If you are learning the code, keep one analysis branch enabled at a time after the first successful run.

5.5 Example 4: cavity direct and adjoint EVP

This case uses a masked, deformed cavity geometry and computes direct and adjoint temporal modes.

Command

```
.venv/bin/python -m examples.example_4_cavity
```

Main knobs:

File excerpt

```
"nx": 241,
"ny": 101,
"omega_target": [1.5, 1.0],
"n_modes": 50,
"ncv": 300,
"beta": 0.0,
"assembly_backend": "scipy"
```

The cavity currently keeps the SciPy assembly path because the adjoint workflow remains SciPy-based.

5.6 Example 5: paraboloid resolvent

This case combines a curvilinear transformation with the axisymmetric model.

Command

```
.venv/bin/python -m examples.example_5_paraboloid
```

Main knobs:

File excerpt

```
"nx": 100,
"ny": 100,
"model": "axy",
"m": 5,
```

```
"omega": 0.0,
"use_omega_shift_fallback": true,
"forcing_zero_vars": ["T", "rho"]
```

6. Run the Martini parity checks

After the example artifacts exist, compare Python outputs with extracted MATLAB bundles:

Command

```
.venv/bin/python validation/martini2024/tools/compare_martini_examples.py
```

The machine-readable registry is:

Expected output

```
validation/martini2024/tools/martini_cases.jsonc
```

The current reports are written under:

Expected output

```
validation/martini2024/reports/martini_parity_latest.json
validation/martini2024/reports/martini_parity_table.csv
```

Warning

If the MATLAB NPZ bundles are absent, the comparison reports missing-bundle failures. That is a data-availability result, not a reason to loosen tolerances or edit tests.

For a complete example pass, use the bundled runner:

Command

```
bash run_all_examples.sh
```

7. Song 2023 validation baseline

The Song 2023 case is a validation workflow for an oblique shock / boundary-layer interaction. It is larger than the Martini examples and should be treated as a study, not as a quick smoke test.

The main files are:

Expected output

```
validation/song_2023/song_2023.py
validation/song_2023/configs/song_2023.jsonc
validation/song_2023/song_2023.sh
```

Run the baseline through the wrapper:

Command

```
bash validation/song_2023/song_2023.sh
```

The baseline JSONC contains the physical parameters, finite-difference order, stride, preprocessing, boundary conditions, and solver settings.

File excerpt

```
"Re": 1e5,
"Ma": 2.15,
"beta": 19.2,
"fd_order": 2,
"stride_ksi": 1,
"stride_eta": 1,
"run_stability": true,
```

```
"run_resolvent": false
```

Boundary conditions:

File excerpt

```
{"type": "dirichlet", "borders": "lt", "variables": "ruvwT"}
{"type": "dirichlet", "borders": "b",  "variables": "ruvw"}
{"type": "neumann",   "borders": "b",  "variables": "T"}
{"type": "neumann",   "borders": "r",  "variables": "ruvwT"}
```

Expected canonical outputs:

Expected output

```
validation/song_2023/song_2023.npz
validation/song_2023/song_2023.log
validation/song_2023/song_2023.png
validation/song_2023/song_2023_baseflow.png
validation/song_2023/song_2023_support_direct_modes.png
```

8. Song validation matrices

Use JSONC matrix files when you want to compare solver choices, preprocessing choices, or finite-difference orders without overwriting the baseline case.

Expected output

```
validation/song_2023/configs/song_2023_matrix.jsonc
validation/song_2023/configs/order_convergence.jsonc
validation/song_2023/configs/song_2023_order_preflight.jsonc
validation/song_2023/configs/song_2023_scaling.jsonc
validation/song_2023/configs/fd4_sensitivity.jsonc
validation/song_2023/configs/shift_scan.jsonc
validation/song_2023/configs/shifted_ksp_matrix.jsonc
```

Run the solver/preprocessing matrix:

Command

```
.venv/bin/python -m validation.song_2023.scripts.run_song_2023_matrix
```

Run the finite-difference order preflight before the full order convergence study:

Command

```
.venv/bin/python -m validation.song_2023.scripts.run_song_2023_order_preflight
```

Run the order convergence study only on a machine with enough memory for the configured guard:

Command

```
.venv/bin/python -m validation.song_2023.scripts.run_song_2023_order_convergence
.venv/bin/python validation/song_2023/scripts/plot_order_convergence.py
```

Warning

The full Song order-convergence study can require many gigabytes of memory. Check `memory_guard` in `validation/song_2023/configs/order_convergence.jsonc` before running it.

9. How to adjust a shipped case

For small experiments, copy the closest Python/JSONC pair into a working location. This keeps the shipped examples intact.

Command

```
mkdir -p tutorial/work
cp examples/example_1_blasius.py tutorial/work/my_blasius.py
cp examples/example_1_blasius_smoke.jsonc tutorial/work/my_blasius.jsonc
PYTHONPATH=src:. .venv/bin/python tutorial/work/my_blasius.py
```

Then edit:

Expected output

```
tutorial/work/my_blasius.jsonc
```

Good first changes:

- reduce `nx` and `ny` for fast tests,
- change `n_modes` only after the first successful run,
- change `fd_order` one value at a time,
- keep `assembly_backend: "scipy"` until the case works,
- keep solver settings in JSONC.

Note

The copied Python file uses `SCRIPT.with_suffix(".jsonc")`. That is why `my_blasius.py` reads `my_blasius.jsonc` automatically when the names match.

10. How to start a new case

Use this checklist before writing new package code:

- choose the closest existing case: Blasius for 2-D resolvent, jet/paraboloid for axisymmetric resolvent, streaky/cavity for EVP branches, Song for SWBLI validation;
- copy the Python/JSONC pair into a study directory;
- keep all physical and numerical parameters in the JSONC file;
- load base-flow data through a small loader rather than hardcoded absolute paths;
- define the boundary-condition recipe explicitly;
- run a reduced grid first;
- save `.npz` outputs every run and plot from the saved file;
- add a test or parity check before changing shared operator code.

For MATLAB-backed parity, add a registry entry only after the reference bundle location is known:

Expected output

```
validation/martini2024/tools/martini_cases.jsonc
```

11. Common first-run problems

- If `petsc4py` or `slepc4py` cannot import, rerun `bash install.sh -check`.
- If a run appears to use the wrong Python, use `.venv/bin/python` explicitly.
- If a full example is too slow, run the smoke or reduced-grid variant first.
- If a parity report says a MATLAB NPZ bundle is missing, generate or provide the bundle; do not change tolerances to hide the missing input.
- If a high-order run changes behavior, compare `fd_order` values through a JSONC matrix instead of editing operator code first.

12. Build this tutorial

The tutorial source and Makefile live together:

Expected output

```
tutorial/tutorial.tex
tutorial/Makefile
```

Build from inside the tutorial directory:

Command

```
cd tutorial
make
```

The build writes:

Expected output

```
tutorial/tutorial.pdf
```